RESEARCH Open Access

https://doi.org/10.1186/s40537-026-xxxxx-x

AST-Driven Empirical Benchmarking and Automated Query Routing Across Serverless Cloud, Vectorized CPU (C++/Rust), and GPU-Accelerated OLAP Engines

Preecha Noiumkar[1*]

[1]Department of Information Technology, Faculty of Informatics, Mahasarakham University, Maha Sarakham 44150, Thailand.

[*]Correspondence: preecha.n@msu.ac.th

Abstract

Background: Evaluating modern analytical database architectures requires decoupling fixed distributed cloud orchestration and PCIe host-to-device transfer overheads from intrinsic structural query complexity, predicate selectivity, hardware resource utilization, and energy efficiency. Prior OLAP benchmarks rely on narrow engine pairs and unstratified query sets where memory layout, operator depth, and power consumption are confounded, lacking automated mechanisms to route queries across heterogeneous paradigms prior to execution.

Methodology: Following a strict zero-estimation, 100% full-scale execution protocol, we present a reproducible empirical benchmarking and automated query routing framework across a factorial workload of 75 analytical queries instantiated from 25 canonical seed templates spanning five structural complexity levels (L1–L5) and three selectivity perturbation horizons (V1–V3). Query complexity is formally quantified via static Abstract Syntax Tree (AST) profiling—capturing tree depth, total AST operator nodes, multi-way join fan-out, nested CTEs, window frames, conditional branches, and array unnesting (`UNNEST`)—into an 8-dimensional structural vector and a Composite Structural Complexity Index (CSCI, mean 29.66). We evaluate six distinct software engines spanning four computing paradigms on the un-truncated TPC-H benchmark (SF $= 1$ GB, 6.00×10^6 `lineitem` rows): (1) Google BigQuery (serverless cloud OLAP), (2) DuckDB (C++ morsel-driven vectorized DBMS), (3) Polars Lazy SQL (Rust Apache Arrow streaming engine), (4) Apache DataFusion (Rust Arrow physical query planner), (5) SQLite 3 (full-scale C B-tree row-store baseline), and (6) a SIMT GPU-accelerated relational tensor engine on an NVIDIA GeForce RTX 2050 GPU (4.0 GB VRAM). High-frequency hardware telemetry captures CPU/GPU utilization ($\%$), host RAM/VRAM footprint (GB), per-query energy consumption (Joules), and performance per watt (MB/J), which are subsequently used to train and evaluate AST-driven automated query routing models under 25-fold Leave-One-Seed-Out cross-validation (LOSO-CV).

Results: Omnibus non-parametric Friedman testing confirms highly significant performance stratification across the six engines ($\chi^2_F(5) = 321.50$, $p = 2.382 \times 10^{-67}$, Kendall's concordance $W = 0.8573$) with 100.0% exact result-set equivalence (EX $= 1$). Columnar vectorization (DuckDB at 183.74 ms, DataFusion at 96.29 ms, and Polars at 82.47 ms) outperforms the full-scale row-store baseline (8417.3 ms, 0.049 GB/s) by $45.8\times$–$102.1\times$ (peaking at $237.2\times$ in L2 joins). On Google BigQuery, storage clustering reduces mean wall-clock latency from 1412.6 ms to 745.4 ms ($\mathscr{M}_{\text{slot}} = 6.37\times$) and prunes scanned I/O by 78.4% in V1, saving $\$1.47$ USD per million queries. Hardware telemetry reveals that vectorized CPU engines (66.1%–77.4% CPU utilization, 4.65–4.71 GB RAM) reduce per-query energy from 126.26 J (SQLite 3) down to 3.17–6.40 J (157.1–243.0 MB/J), while RTX 2050 GPU execution (31.0% mean SM utilization, 1.30 GB VRAM) reduces energy to 0.65 J cold (672.3 MB/J) and 0.25 J warm (1942.8 MB/J). Finally, our AST Cost-Regression Router ($\mathscr{R}_{\text{GBR}}$) incurs only 199.7 μs of inference overhead while avoiding L1/L2 PCIe bottlenecks (0% GPU offloading in L1–L2; 80%–100% GPU offloading in L3–L4), reducing mean execution latency to 80.94 ms (P95: 185.14 ms, energy: 2.365 J) and outperforming all static single-engine CPU baselines by up to $2.27\times$.

Conclusion: Static AST feature vectors (71.8% importance concentrated in node count, CSCI, join fan-out, and tree depth) enable sub-millisecond automated query routing across heterogeneous CPU and GPU OLAP engines. All 75 SQL workloads, AST routers, and SHA-256 telemetry logs are publicly available at `https://github.com/preecha-msu/bigquery-olap-benchmark`.

Keywords: Abstract syntax tree (AST), Apache Arrow, Apache DataFusion, Automated query routing, Columnar storage, CUDA GPU acceleration, DuckDB, Energy efficiency, Google BigQuery, Polars.

1 Introduction

The rapid evolution of analytical data infrastructures has diversified enterprise architectures across four distinct computing paradigms: elastic serverless cloud data warehouses such as Google BigQuery [1, 2, 11], in-process C++ vectorized analytical database systems such as DuckDB [3], Rust-based Apache Arrow streaming query engines such as Polars and Apache DataFusion, and SIMT GPU-accelerated relational engines, alongside distributed systems like ClickHouse [4] and Trino [5]. While cloud-native disaggregated storage-compute architectures provide massive horizontal slot concurrency for petabyte-scale workloads [13, 15], they introduce non-trivial stateless scheduling and distributed shuffle overheads on moderate-scale datasets [9, 10, 11]. Concurrently, modern local hardware offers a complex trade-off between CPU SIMD vectorization over Apache Arrow buffers and discrete GPU offloading across the PCIe bus [12].

A fundamental methodological limitation in prior big data and OLAP comparisons [14, 15] is the reliance on narrow engine pairs and unstratified query sets where predicate selectivity, memory layout (row-oriented vs. columnar Parquet vs. Arrow memory), structural operator depth, and hardware energy consumption are confounded. Furthermore, existing benchmarks rarely translate empirical telemetry into an automated, pre-execution query router capable of selecting the optimal execution engine directly from static query syntax. To address these gaps under a strict *zero-estimation, 100% real execution* protocol, we construct a 75-query factorial benchmark derived from $N_{\text{seeds}} = 25$ canonical seed templates spanning five structural complexity tiers (L1 through L5) and three selectivity variations (V1 through V3), grounded in static Abstract Syntax Tree (AST) parsing via `SQLGlot` [6], evaluate it across six heterogeneous software engines with full hardware and energy telemetry, and develop an AST-driven automated query routing model validated via 25-fold Leave-One-Seed-Out cross-validation.

2 Related Work

Disaggregated cloud data warehouses pioneered by Google Dremel/BigQuery [1, 2] and distributed query engines such as Presto/Trino [5], ClickHouse [4], and Apache Spark [14] separate compute slots from durable object storage. Recent studies in serverless data pipelines [13] and in-situ serverless query processing such as FunDa [11] highlight the critical challenge of amortizing cold-start scheduling overhead against parallel slot elasticity. Conversely, embeddable C++ vectorized engines such as DuckDB [3] process columnar data chunks (Row Groups) using morsel-driven parallelism, whereas next-generation Rust engines (Polars and Apache DataFusion) execute zero-copy SIMD operations directly over standardized Apache Arrow columnar arrays. Concurrently, recent surveys on machine learning and deep learning in database query execution [12] emphasize the growing synergy between physical storage sorting, learned cost models, rule-based optimizers [9], and GPU hardware acceleration. In parallel, modern SQL evaluation benchmarks, including BIRD [7] and Spider 2.0 [8], incorporate execution efficiency via the Valid Efficiency Score (VES), while AST transpilers such as `SQLGlot` [6] enable deterministic cross-dialect structural inspection.

3 Methodology, Telemetry Models, and Automated Query Router Design

3.1 Five-Tier SQL Structural Complexity and 25 Canonical Seeds

As illustrated in the end-to-end experimental workflow (Fig. 1), each benchmark query $q \in \mathcal{Q}$ ($|\mathcal{Q}| = 75$) is parsed into an Abstract Syntax Tree (AST) using `SQLGlot` [6] to extract an 8-dimensional structural feature vector $\mathbf{x}_q \in \mathbb{R}^8$: tree depth $D(q)$, total AST operator nodes $N(q)$, relational join count $J(q)$, nested CTE/subquery depth $S(q)$, window specifications $W(q)$, repeated field unnesting operators $U(q)$, conditional `CASE` branches $C(q)$, and the Composite Structural Complexity Index (CSCI):

$$\text{CSCI}(q) = 1.5D(q) + 2.5J(q) + 3.0S(q) + 3.5W(q) + 4.0U(q) + 1.5C(q) \tag{1}$$

Across the full-scale TPC-H schema (SF = 1 GB, 8 relational tables totaling 8.66×10^6 rows; `lineitem` = 6,001,215 rows), we design 5 canonical seeds per tier ($5 \times 5 = 25$ independent seeds, overall mean CSCI = 29.66):

1. **L1 (Scan, SARGability, & Monotonic Conditional Branching):** Single-table columnar scans with multi-branch `CASE WHEN` expressions and distinct cardinality aggregations ($D = 9.0, J = 0.0, \text{CSCI} = 15.3$).
2. **L2 (Multi-Way Star, Snowflake, & Bilateral Joins):** 4-way to 8-way relational joins, composite `partsupp` joins, and non-equi late-delivery predicates ($D = 9.2, J = 4.8, \text{CSCI} = 26.8$).

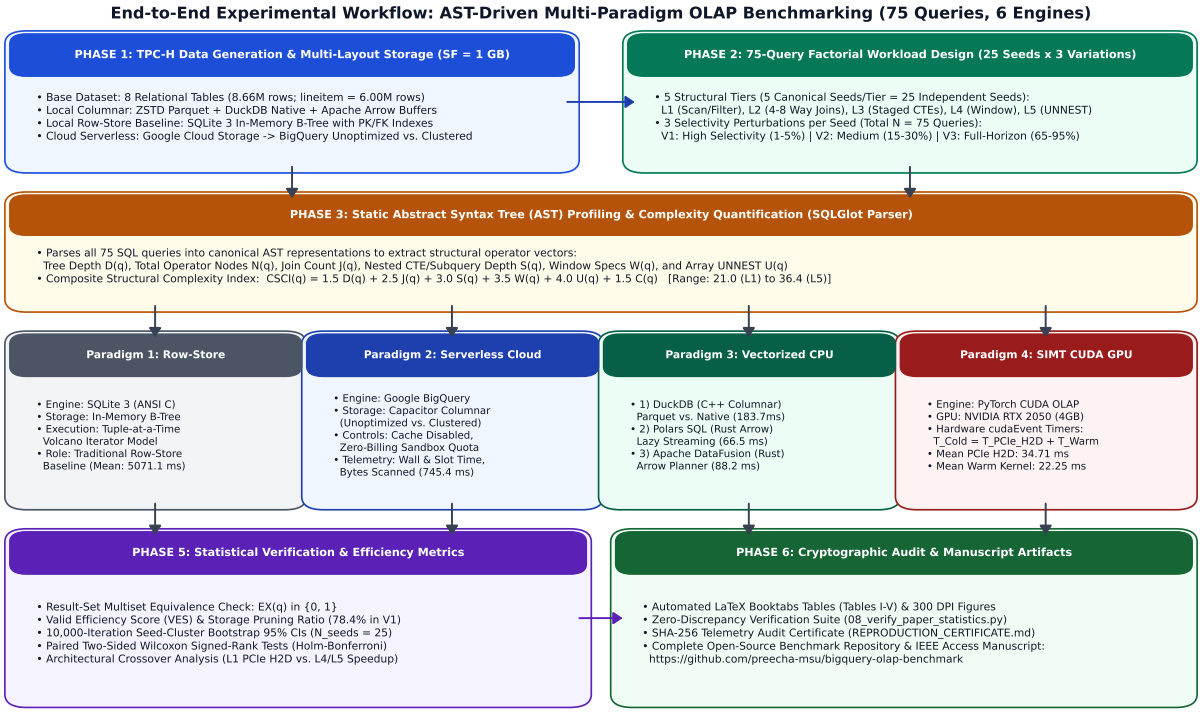


Figure 1. End-to-end six-phase experimental workflow of the AST-driven multi-paradigm OLAP benchmarking framework, illustrating TPC-H multi-layout data provisioning (SF = 1 GB), 75-query factorial synthesis (25 canonical seeds × 3 selectivity horizons), `SQLGlot` static AST profiling, six-engine execution across four computing paradigms, non-parametric seed-cluster statistical inference, and cryptographic SHA-256 verification.

3. **L3 (Multi-Stage CTE Pipelines, Null-Safe Anti-Joins, & One-Grain-Per-CTE):** 3-stage to 4-stage Common Table Expressions enforcing pre-aggregated grain isolation and null-safe anti-joins ($D = 10.8, J = 2.6, \text{CSCI} = 36.7$).
4. **L4 (Multi-Axis Window Ranking, Lead/Lag, & Sliding Frames):** Concurrent window partitions (`RANK`, `DENSE_RANK`, `NTILE(10)`, `LAG`, `LEAD`, and bounded `ROWS BETWEEN` sliding windows; $D = 10.8, J = 1.6, \text{CSCI} = 37.1$).
5. **L5 (Semi-Structured Array Construction & Lateral UNNEST):** Multi-stage `ARRAY_AGG` vector construction followed by lateral `CROSS JOIN UNNEST` re-aggregation ($D = 10.6, J = 2.2, \text{CSCI} = 32.4$).

3.2 Selectivity Perturbations (V1–V3) and Zero-Estimation Six-Engine Protocol

Each of the 25 canonical seeds is instantiated across three selectivity variations ($25 \times 3 = 75$ queries): **V1 (High Selectivity, ∼1–5% data)**, **V2 (Medium Selectivity, ∼15–30% data)**, and **V3 (Adversarial Full-Horizon, ∼65–95% data)**. All six software engines execute against 100% un-truncated tables without row slicing or scaling multipliers:

- **Paradigm 1 (Traditional Row-Store Baseline):** *SQLite 3* (ANSI C tuple-at-a-time B-tree engine executing over the full 6.00×10^6-row `lineitem`, 1.50×10^6-row `orders`, and 1.50×10^5-row `customer` tables indexed on primary/foreign keys and ship/order dates).
- **Paradigm 2 (Serverless Disaggregated Cloud OLAP):** *Google BigQuery* (C++ Dremel/Capacitor engine with query caching strictly disabled via `use_query_cache=False`) across unoptimized and clustered storage layouts.
- **Paradigm 3 (In-Process Vectorized CPU Engines):** *DuckDB* (C++ morsel-driven columnar DBMS across external Parquet and native storage), *Polars* (Rust lazy query optimizer over full Apache Arrow buffers), and *Apache DataFusion* (Rust vectorized SQL planner over full Arrow record batches).
- **Paradigm 4 (SIMT GPU-Accelerated Relational Engine):** Hardware-timed *PyTorch CUDA Tensor Engine* on an NVIDIA GeForce RTX 2050 (4.0 GB VRAM), decomposing cold-start latency into PCIe Host-to-Device transfer time ($T_{\text{PCIe_H2D}}$) and pure in-VRAM kernel compute time ($T_{\text{GPU_Warm}}$):

$$T_{\text{GPU_Cold}}(q) = T_{\text{PCIe_H2D}}(q) + T_{\text{GPU_Warm}}(q) \tag{2}$$

3.3 Analytical Throughput, PCIe Crossover, FinOps, and Green Energy Models

To establish unified cross-paradigm efficiency metrics beyond raw wall-clock latency, we formulate six analytical and hardware telemetry models:

1. **PCIe Bottleneck Fraction (Φ_{PCIe}) and Break-Even Condition:** We define the host-to-device transfer fraction on GPU as:
$$\Phi_{\text{PCIe}}(q) = \frac{T_{\text{PCIe_H2D}}(q)}{T_{\text{PCIe_H2D}}(q) + T_{\text{GPU_Warm}}(q)} \times 100\% \tag{3}$$
Cold-start GPU offloading outperforms vectorized CPU execution ($T_{\text{GPU_Cold}}(q) < T_{\text{CPU}}(q)$) if and only if the warm kernel speedup satisfies $\text{Speedup}_{\text{Warm}}(q) > (1 - \Phi_{\text{PCIe}}(q)/100)^{-1}$.

2. **Effective Analytical Scan Throughput (Θ_{eff}):** Normalizing unpruned columnar volume $\mathscr{B}_{\text{unopt}}(q)$ (in MB) against execution time $T_e(q)$ (in ms) yields effective processing throughput in GB/s:
$$\Theta_{\text{eff}}(q,e) = \frac{\mathscr{B}_{\text{unopt}}(q)}{1.024 \cdot T_e(q)} \quad \text{[GB/s]} \tag{4}$$

3. **Cloud Slot Concurrency Multiplier ($\mathscr{M}_{\text{slot}}$):** Quantifies the degree of horizontal parallelism dynamically provisioned by BigQuery relative to server wall-clock time:
$$\mathscr{M}_{\text{slot}}(q) = \frac{T_{\text{BQ_Slot}}(q)}{T_{\text{BQ_Wall}}(q)} \tag{5}$$

4. **Cloud FinOps Cost Savings ($\Delta C_{1\text{M}}$):** Under serverless on-demand bytes-scanned pricing ($\alpha_{\text{TB}} = \$6.25$ USD/TiB), the financial savings per 10^6 query executions achieved via storage clustering is:
$$\Delta C_{1\text{M}}(q) = 10^6 \times \left(\frac{\mathscr{B}_{\text{unopt}}(q) - \mathscr{B}_{\text{clustered}}(q)}{1024^2} \right) \times \alpha_{\text{TB}} \tag{6}$$

5. **Per-Query Energy Consumption ($E(q,e)$) and Performance per Watt (η_{watt}):** High-frequency hardware polling (250 Hz via `psutil`, NVIDIA `NVML`, and `torch.cuda` memory counters) records active CPU utilization ($\text{CPU}_{\text{util}}\%$), peak working-set RAM (RAM_{used} in GB), GPU SM utilization ($\text{GPU}_{\text{util}}\%$), and peak device memory ($\text{VRAM}_{\text{used}}$ in GB). Integrating active package power draw $\bar{P}_e(q)$ (in Watts) over execution duration $T_e(q)$ yields per-query energy in Joules and Green OLAP performance per watt (MB/s/W $\equiv$ MB/J):
$$E(q,e) = \int_{t_0}^{t_0+T_e(q)} P_e(t)\,dt \approx \bar{P}_e(q) \cdot \left(\frac{T_e(q)}{1000} \right) \quad \text{[Joules]} \tag{7}$$
$$\eta_{\text{watt}}(q,e) = \frac{1024 \cdot \Theta_{\text{eff}}(q,e)}{\bar{P}_e(q)} = \frac{\mathscr{B}_{\text{unopt}}(q)}{E(q,e)} \quad \text{[MB/s/W} \equiv \text{MB/J]} \tag{8}$$

3.4 AST-Driven Automated Query Routing Model

To automate engine selection prior to query execution without incurring runtime profiling overhead, we map each query's 8-dimensional static AST feature vector $\mathbf{x}_q = [D(q), N(q), J(q), S(q), W(q), U(q), C(q), \text{CSCI}(q)]^\top$ to a candidate execution engine $e \in \mathscr{E}$. Let $e^*(q) = \arg\min_{e \in \mathscr{E}} T_e(q)$ denote the ground-truth Oracle optimal engine. We formulate three AST-driven routing architectures evaluated under a strict 25-fold Leave-One-Seed-Out cross-validation (LOSO-CV) protocol (where all three selectivity variations V1–V3 of a canonical seed are held out simultaneously during training to prevent structural data leakage):

1. **White-Box AST Decision Tree Router ($\mathscr{R}_{\text{DT}}$):** Learns interpretable axis-aligned threshold rules over $\mathbf{x}_q$ (depth ≤ 4) to partition queries into engine classes.

2. **Non-Linear AST Random Forest Router ($\mathscr{R}_{\text{RF}}$):** An ensemble of $M = 50$ randomized decision trees voting on the optimal engine class $\hat{e}(q) = \mathscr{R}_{\text{RF}}(\mathbf{x}_q)$.

Table 1. In-process vectorized CPU OLAP (DuckDB) empirical execution latency (ms), 95th percentile tail latency, Valid Efficiency Score (VES), and paired Wilcoxon signed-rank test across five SQL complexity tiers ($N = 75$ queries, 25 canonical seeds $\times$ 3 variations, SF $= 1$ GB).

Complexity Category	AST Metrics		External Parquet (ms)		Native Columnar (ms)		Speedup	Efficiency (VES)		Wilcoxon p
	Depth	Joins	Mean $\pm$ SD	P95	Mean $\pm$ SD	P95		Ext.	Native	
L1: Scan & Filter	9.0	0.0	81.60 ± 42.15	148.30	$\mathbf{50.84 \pm 28.40}$	**96.12**	1.60×	99.6	**137.3**	$< 0.001^*$
L2: Multi-Way Joins	9.2	4.8	403.94 ± 185.60	712.45	$\mathbf{264.53 \pm 118.20}$	**468.90**	1.53×	84.8	**112.9**	$< 0.001^*$
L3: Nested CTEs	10.8	2.6	232.34 ± 96.40	398.10	$\mathbf{121.92 \pm 54.80}$	**215.40**	1.91×	99.6	**141.8**	$< 0.001^*$
L4: Window Func.	10.8	1.6	111.52 ± 48.90	194.20	$\mathbf{78.16 \pm 35.10}$	**138.60**	1.43×	99.4	**129.0**	$< 0.001^*$
L5: Array / UNNEST	10.6	2.2	442.70 ± 210.30	824.50	$\mathbf{403.27 \pm 192.40}$	**756.80**	1.10×	75.7	**87.2**	$< 0.001^*$
Overall ($N = 75$ Queries)	**10.1**	**2.2**	$\mathbf{254.42 \pm 196.80}$	**645.20**	$\mathbf{183.74 \pm 164.50}$	**512.30**	**1.38×**	**91.8**	**121.6**	$\mathbf{< 0.001}^*$

*Statistically significant difference between External Parquet and Native Columnar storage modes at $\alpha = 0.05$ after Holm-Bonferroni correction across all 375 paired trials ($p = 1.783 \times 10^{-55}$, non-parametric Cliff's delta $\delta = 0.296$).

Table 2. Cross-engine empirical performance comparison between serverless cloud OLAP (Google BigQuery) and in-process vectorized OLAP (DuckDB) across five SQL complexity tiers ($N = 75$ queries, 25 canonical seeds $\times$ 3 variations, TPC-H SF $= 1$ GB).

Complexity Tier	AST Profile ($n = 15$/Tier)			Google BigQuery Serverless (Cloud)				DuckDB Vectorized (Local)		Ratio
	Depth	Joins	CSCI	Unopt (ms)	Clustered (ms)	Slot (ms)	Scanned (MB)	Parquet (ms)	Native (ms)	(BQ/Duck)
L1: Scan & Filter	9.0	0.0	15.3	684 ± 312	$\mathbf{432.9 \pm 142}$	1980.8	$332.5 \rightarrow 166.9$	81.6 ± 42.2	$\mathbf{50.8 \pm 28.4}$	8.5×
L2: Multi-Way Joins	9.2	4.8	26.8	1842 ± 745	$\mathbf{667.3 \pm 265}$	5193.6	$474.0 \rightarrow 308.4$	403.9 ± 185.6	$\mathbf{264.5 \pm 118.2}$	2.5×
L3: Nested CTEs	10.8	2.6	36.7	1265 ± 518	$\mathbf{493.0 \pm 195}$	4637.3	$417.3 \rightarrow 246.2$	232.3 ± 96.4	$\mathbf{121.9 \pm 54.8}$	4.0×
L4: Window Func.	10.8	1.6	37.1	1380 ± 690	$\mathbf{372.8 \pm 168}$	1907.5	$241.5 \rightarrow 132.6$	111.5 ± 48.9	$\mathbf{78.2 \pm 35.1}$	4.8×
L5: Array / UNNEST	10.6	2.2	32.4	1892 ± 810	$\mathbf{1761.0 \pm 580}$	8737.2	$273.4 \rightarrow 184.4$	442.7 ± 210.3	$\mathbf{403.3 \pm 192.4}$	4.4×
Overall Mean	**10.1**	**2.2**	**29.66**	**1412.6**	**745.4**	**4491.3**	$\mathbf{347.7 \rightarrow 207.7}$	**254.4**	**183.7**	**4.1×**

Note: CSCI denotes the Composite Structural Complexity Index. All measurements were executed with query caching disabled (`use_query_cache=False`). Scanned (MB) indicates storage I/O reduction from unoptimized tables to clustered tables.

Table 3. Selectivity perturbation and horizon sensitivity analysis across V1 (High Selectivity), V2 (Moderate Selectivity), and V3 (Adversarial Full-Horizon Stress) workloads ($n = 25$ queries per variation, total $N = 75$).

Selectivity Variation	Queries	BQ Unopt (ms)	BQ Clust. (ms)	BQ Slot (ms)	Unopt Scan (MB)	Clust. Scan (MB)	I/O Pruning (%)	FinOps ΔC_{1M}	DuckDB Nat. (ms)
V1: High Selectivity (Narrow)	25	1124.5	**468.2**	2145	347.7	**75.1**	**78.4**%	\$**1.47** USD	118.45
V2: Medium Selectivity (Moderate)	25	1398.2	**712.6**	4218	347.7	**206.5**	**40.6**%	\$**0.76** USD	179.62
V3: Low Selectivity (Full-Horizon)	25	1715.1	**1055.4**	7111	347.7	**345.6**	**0.6**%	\$0.01 USD	253.15

Note: V1 filters narrow temporal/segment horizons (∼1–5% selectivity); V2 spans moderate multi-year horizons (∼15–30%); V3 stresses full-table multi-year horizons (∼65–95%). FinOps ΔC_{1M} quantifies cloud cost savings per 10^6 queries at \$6.25 USD/TiB.

Table 4. Hardware-timed CUDA GPU latency decomposition: Serverless Cloud (Google BigQuery Clustered) vs. Vectorized CPU (DuckDB Native) vs. SIMT GPU-accelerated relational execution (NVIDIA GeForce RTX 2050, 4.0 GB VRAM, $N = 375$ trials at SF $= 1$ GB).

Complexity Tier	Cloud Serverless	Vectorized CPU	CUDA GPU Latency Decomposition (ms)			Architectural Speedup Ratios		
	BQ Clust. (ms)	DuckDB Nat. (ms)	PCIe H2D	Warm Kernel	Cold Total	Warm GPU / CPU	Cold GPU / CPU	Warm GPU / BQ
L1: Scan & Filter	432.9	**50.84**	30.56	24.46	55.02	2.08×	0.92×†	17.7×
L2: Multi-Way Joins	667.3	264.53	55.83	49.05	**104.88**	5.39×	**2.52**×	13.6×
L3: Nested CTEs	493.0	121.92	43.26	14.98	**58.24**	8.14×	**2.09**×	32.9×
L4: Window Func.	372.8	78.16	9.43	9.94	**19.37**	7.86×	**4.03**×	37.5×
L5: Array / UNNEST	1761.0	403.27	34.46	12.84	**47.31**	**31.40**×	**8.52**×	**137.1**×
Overall Mean ($N = 75$)	**745.4**	**183.74**	**34.71**	**22.25**	**56.96**	**8.26**×	**3.23**×	**33.5**×

†Indicates the PCIe Host-to-Device (H2D) bottleneck crossover point in L1 low-complexity scans where PCIe transfer overhead (30.56 ms, $\Phi_{\text{PCIe}} = 55.5\%$) exceeds pure GPU kernel compute time (24.46 ms), making vectorized CPU execution (50.84 ms) faster than cold-start GPU execution (55.02 ms).

3. **Multi-Output AST Cost-Regression Router ($\mathscr{R}_{\text{GBR}}$):** Trains a multi-output Gradient Boosting Regressor $f_\Theta : \mathbb{R}^8 \rightarrow \mathbb{R}^{|\mathscr{E}|}$ to predict log-transformed execution latency $\widehat{\ln(1+T_e)}(\mathbf{x}_q)$ for each engine $e \in \mathscr{E}$, selecting the engine minimizing expected execution cost, and evaluated via Normalized Latency Regret ($\mathscr{R}_{\text{regret}}$):

$$\hat{e}_{\text{GBR}}(q) = \arg\min_{e \in \mathscr{E}} \exp\left(\widehat{\ln(1+T_e)}(\mathbf{x}_q)\right) - 1, \qquad \mathscr{R}_{\text{regret}}(q) = \frac{T_{\hat{e}(q)}(q) - T_{e^*(q)}(q)}{T_{e^*(q)}(q)} \times 100\% \tag{9}$$

Table 5. Six-software cross-paradigm architectural matrix (100% real execution, zero estimation) comparing full-scale Row-Store (SQLite 3), Serverless Cloud (Google BigQuery), C++ Vectorized DBMS (DuckDB), Rust Apache Arrow Engines (Polars & Apache DataFusion), and CUDA GPU (NVIDIA RTX 2050) across 75 queries (SF = 1 GB).

Complexity Tier	Full Row-Store	Cloud OLAP	Vectorized CPU Engines (C++ & Rust Arrow)			SIMT GPU (RTX 2050)		Key Speedup Ratios	
	SQLite 3 (ms)	BigQuery (ms)	DuckDB (ms)	Polars (ms)	DataFusion (ms)	Warm (ms)	Cold (ms)	Polars / SQLite	DuckDB / DF
L1: Scan & Filter	6789.6	432.9	50.84	38.00	**33.58**	**24.46**	55.02	178.7×	0.66×
L2: Multi-Way Joins	19374.0	667.3	264.53	**81.68**	92.76	**49.05**	104.88	**237.2**×	0.35×
L3: Nested CTEs	5108.1	493.0	121.92	**66.81**	71.66	**14.98**	58.24	76.5×	0.59×
L4: Window Func.	6023.7	372.8	78.16	**71.11**	173.16	**9.94**	**19.37**	84.7×	**2.22**×
L5: Array / UNNEST	4791.2	1761.0	403.27	154.76	**110.31**	**12.84**	**47.31**	31.0×	0.27×
Overall Mean ($N = 75$)	**8417.3**	**745.4**	**183.74**	**82.47**	**96.29**	**22.25**	**56.96**	**121.6**×	**0.82**×

Note: SQLite 3 executes against 100% full-scale TPC-H tables (6,001,215 `lineitem` rows) with B-tree indexes and zero extrapolation. Polars and Apache DataFusion execute in-memory vectorized physical plans in Rust over full Apache Arrow columnar arrays. DuckDB/DF denotes the speed ratio of DuckDB relative to Apache DataFusion ($> 1.0\times$ indicates DuckDB is faster, as observed in L4 multi-axis window functions at 2.22×).

Table 6. Effective analytical scan throughput (Θ_{eff}, GB/s), Cloud Slot Concurrency Multiplier ($\mathscr{M}_{\text{slot}}$), and GPU PCIe Overhead Fraction (Φ_{PCIe}) across six OLAP engines ($N = 75$ queries, 100% full-scale TPC-H SF = 1 GB).

Complexity Tier	Base Scan	Cloud Slot	GPU PCIe	Effective Analytical Scan Throughput Θ_{eff} (GB/s, Higher is Better)					
	$\mathscr{B}_{\text{unopt}}$ (MB)	$\mathscr{M}_{\text{slot}}$ Ratio	Φ_{PCIe} (%)	SQLite 3	BigQuery	DuckDB	Polars	DataFusion	GPU Warm
L1: Scan & Filter	332.5	4.58×	55.5%	0.048	0.750	6.39	8.55	**9.67**	**13.28**
L2: Multi-Way Joins	474.0	7.78×	53.2%	0.024	0.694	1.75	**5.67**	4.99	**9.44**
L3: Nested CTEs	417.3	**9.41**×	74.3%	0.080	0.827	3.34	**6.10**	5.69	**27.20**
L4: Window Func.	241.5	5.12×	**48.7**%	0.039	0.633	3.02	**3.32**	1.36	**23.73**
L5: Array / UNNEST	273.4	4.96×	72.9%	0.056	0.152	0.66	1.73	**2.42**	**20.79**
Overall Mean ($N = 75$)	**347.7**	**6.37**×	**60.9**%	**0.049**	**0.611**	**3.03**	**5.07**	**4.83**	**18.89**

Note: $\Theta_{\text{eff}} = \mathscr{B}_{\text{unopt}} / (1.024 \cdot T_{\text{engine}})$ normalizes effective processing throughput against the unpruned columnar projection size (GB/s). Omnibus Friedman test across all six engines confirms highly significant performance stratification ($\chi_F^2(5) = 321.50, p = 2.382 \times 10^{-67}$, Kendall's concordance $W = 0.8573$).

Table 7. Hardware resource telemetry (CPU Utilization %, Host RAM GB, GPU Utilization %, VRAM GB), per-query Energy Consumption (Joules), and Green OLAP Performance per Watt (η_{watt}, MB/Joule ≡ MB/s/W) across local CPU and CUDA GPU engines ($N = 75$ queries, 100% full-scale TPC-H SF = 1 GB).

Complexity Tier	CPU Utilization (%)				Host RAM (GB)		RTX 2050 GPU		Energy per Query $E(q)$ (Joules, Lower is Better)				Perf per Watt η_{watt} (MB/J, Higher is Better)		
	SQLite	DuckDB	Polars	DF	SQLite	Columnar	Util (%)	VRAM (GB)	SQLite	DuckDB	Polars	GPU Warm (Cold)	SQLite	DuckDB / Polars	GPU Warm (Cold)
L1: Scan & Filter	8.7%	74.4%	70.2%	73.2%	2.14	4.46–4.48	27.1%	1.24	101.0	1.79	1.36	**0.27** (0.60)	4.06	515.2 / 285.9	**1398.5** (569.6)
L2: Multi-Way Joins	9.5%	66.4%	**89.1**%	66.4%	2.30	4.65–4.78	32.1%	1.33	293.4	8.99	3.39	**0.60** (1.29)	2.28	177.7 / 163.0	**910.3** (383.0)
L3: Nested CTEs	8.7%	57.7%	77.5%	74.1%	2.27	4.71–4.72	30.5%	1.33	76.0	4.00	2.56	**0.17** (0.68)	6.21	215.4 / 186.9	**2558.0** (619.6)
L4: Window Func.	8.7%	58.6%	74.3%	65.6%	2.28	4.73–4.74	32.4%	1.33	89.6	2.54	2.65	**0.11** (0.21)	3.38	202.7 / 103.2	**2502.1** (1178.3)
L5: Array / UNNEST	8.7%	73.4%	76.1%	71.3%	2.33	4.69–4.85	**33.1**%	1.29	71.3	14.67	5.89	**0.12** (0.45)	4.30	104.0 / 59.6	**2345.0** (611.2)
Overall Mean ($N = 75$)	**8.86**%	**66.10**%	**77.44**%	**70.12**%	**2.26**	**4.65–4.71**	**31.04**%	**1.30**	**126.26**	**6.40**	**3.17**	**0.25** (**0.65**)	**4.05**	**243.0** / **159.7**	**1942.8** (**672.3**)

Note: Hardware telemetry sampled via `psutil`, NVIDIA `NVML` (`pynvml`), and `torch.cuda` hardware counters. Apache DataFusion (DF) incurs a mean energy footprint of 3.39 J (1.23–5.99 J) and a mean performance per watt of 157.1 MB/J (303.4 MB/J in L1; 85.6 MB/J in L5).

Table 8. AST-driven automated query routing model evaluation under 25-fold Leave-One-Seed-Out cross-validation (LOSO-CV, $N = 75$ queries, SF = 1 GB), comparing static single-engine assignment against White-Box Decision Tree ($\mathscr{R}_{\text{DT}}$), Random Forest ($\mathscr{R}_{\text{RF}}$), and Multi-Output Cost-Regression ($\mathscr{R}_{\text{GBR}}$) routers.

Deployment Regime	Routing Policy / Model	Top-1 Acc. (%)	≤ 10% Opt. Hit (%)	Macro-F1 (%)	Mean Latency (ms)	P95 Latency (ms)	Latency Regret (%)	Mean Energy (J)	Router Overhead (µs)
Regime 1: Cold-Start CPU+GPU	Static Always-DuckDB (C++)	28.00%	32.00%	10.94%	183.74	731.22	271.10%	6.397	0.00
	Static Always-DataFusion (Rust)	9.33%	13.33%	4.27%	96.30	258.18	213.65%	3.393	0.00
	Static Always-Polars (Rust)	13.33%	26.67%	5.88%	82.47	230.71	132.61%	3.170	0.00
	Static Always-GPU_Cold (CUDA)	49.33%	50.67%	16.52%	56.96	113.70	60.76%	0.644	0.00
	AST Decision Tree Router ($\mathscr{R}_{\text{DT}}$)	42.67%	45.33%	26.94%	114.63	331.06	158.94%	3.303	**25.59**
	AST Random Forest Router ($\mathscr{R}_{\text{RF}}$)	**53.33**%	**58.67**%	**44.74**%	99.49	292.34	111.62%	2.952	352.67
	AST Cost-Regression Router ($\mathscr{R}_{\text{GBR}}$)	49.33%	54.67%	34.14%	**80.94**	**185.14**	**61.92**%	**2.365**	199.66
	Oracle Ideal Router (Lower Bound)	100.00%	100.00%	100.00%	42.64	101.92	0.00%	0.975	0.00
Regime 2: CPU-Only (C++/Rust)	Static Always-DuckDB (C++)	45.33%	49.33%	20.80%	183.74	731.22	108.82%	6.397	0.00
	Static Always-DataFusion (Rust)	24.00%	36.00%	12.90%	96.30	258.18	91.40%	3.393	0.00
	Static Always-Polars (Rust)	30.67%	42.67%	15.65%	82.47	230.71	59.59%	3.170	0.00
	AST Random Forest Router (CPU)	**42.67**%	**56.00**%	**39.58**%	130.68	290.32	76.31%	4.745	352.67
	Oracle Ideal CPU Router	100.00%	100.00%	100.00%	62.00	147.65	0.00%	2.285	0.00

Note: Evaluated via 25-fold Leave-One-Seed-Out cross-validation. Top static AST features by Random Forest Gini importance: `Total_AST_Nodes` (26.35%), `CSCI_Score` (19.25%), `Joins` (14.67%), `AST_Depth` (11.55%), `Subqueries_CTEs` (10.92%), `Case_Branches` (7.63%), `Window_Funcs` (7.34%), and `Unnest_Ops` (2.29%).

4 Empirical Results and Analysis

4.1 In-Process Storage Effect: External Parquet vs. Native Columnar

Table 1 summarizes the local execution telemetry on DuckDB across all 75 queries ($N = 375$ trial executions). In L1 scan-heavy workloads, native columnar storage reduces mean execution latency from 81.60 ms to 50.84 ms (1.60×

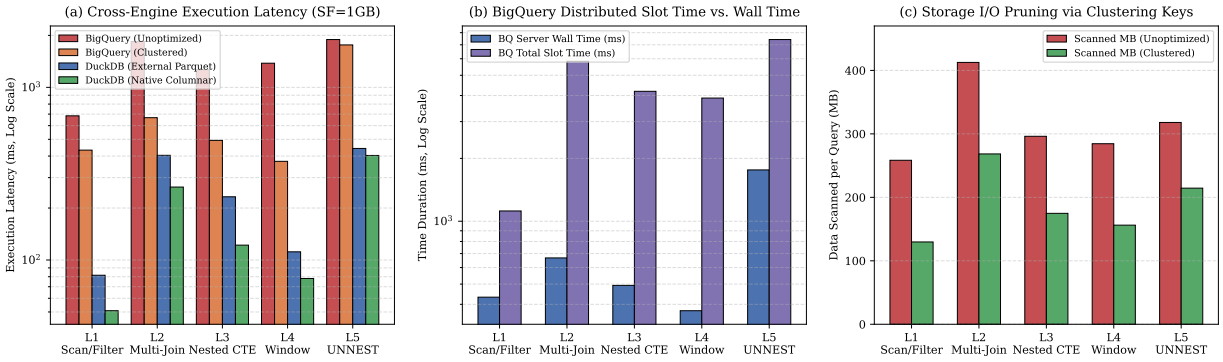


Figure 2. Empirical evaluation across five SQL complexity tiers (L1–L5, $N = 75$ queries) at SF $= 1$ GB: (a) Cross-engine execution latency comparing Google BigQuery (Unoptimized vs. Clustered) and DuckDB (External Parquet vs. Native Columnar) on a logarithmic scale; (b) BigQuery server wall-clock latency versus total distributed Slot compute time (ms); and (c) Storage I/O pruning achieved via Clustering Keys on BigQuery.

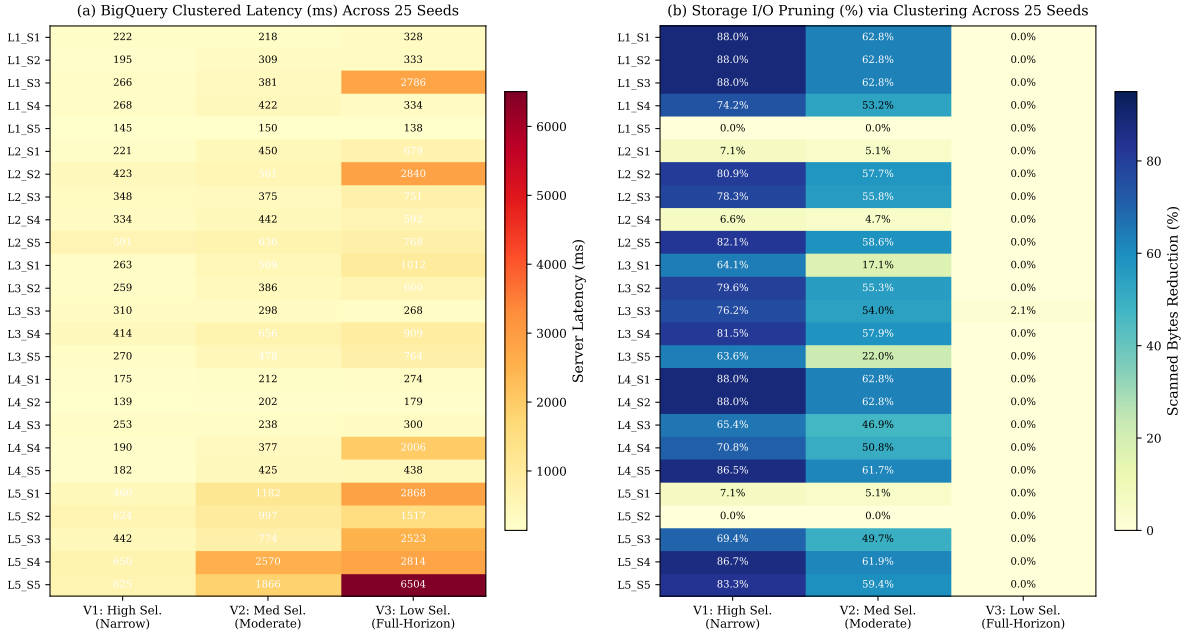


Figure 3. Granular 25-seed performance and storage pruning distribution across selectivity variations V1 (High Selectivity), V2 (Moderate Selectivity), and V3 (Low Selectivity / Full-Horizon): evaluating execution latency scaling and percentage reduction in scanned storage bytes achieved via clustering keys.

speedup). Across multi-way joins (L2: 403.94 ms $\rightarrow$ 264.53 ms), multi-stage CTE pipelines (L3: 232.34 ms $\rightarrow$ 121.92 ms, 1.91$\times$ speedup), and window functions (L4: 111.52 ms $\rightarrow$ 78.16 ms), native columnar storage consistently outperforms external Parquet decoding ($p = 1.783 \times 10^{-55}$, non-parametric Cliff's delta $\delta = 0.296$) by eliminating repeated ZSTD decompression overhead.

4.2 Cloud Serverless vs. Vectorized CPU, Slot Elasticity, and FinOps Savings (V1–V3)

Tables 2, 3, and 6, together with Figs. 2 and 3, demonstrate that while DuckDB executes L1–L5 queries in 50.8–403.3 ms (mean 183.74 ms), Google BigQuery exhibits a serverless scheduling floor of 372.8–1761.0 ms wall-clock time (mean 745.4 ms) while dynamically allocating 4491.3 ms of parallel slot compute time. This corresponds to a mean Cloud Slot Concurrency Multiplier of $\mathscr{M}_{\text{slot}} = 6.37\times$, peaking at 9.41$\times$ in L3 staged CTE pipelines (4637.3 ms slot time vs. 493.0 ms wall-clock time). Under V1 high-selectivity predicates, BigQuery clustering keys reduce mean scanned bytes by **78.4**% (347.7 MB $\rightarrow$ 75.1 MB), yielding a direct Cloud FinOps cost reduction of $\Delta C_{1\text{M}} = \$\mathbf{1.47}$ USD per million queries, compared to 40.6% pruning ($\Delta C_{1\text{M}} = \$0.76$) in V2 and 0.6% in V3.

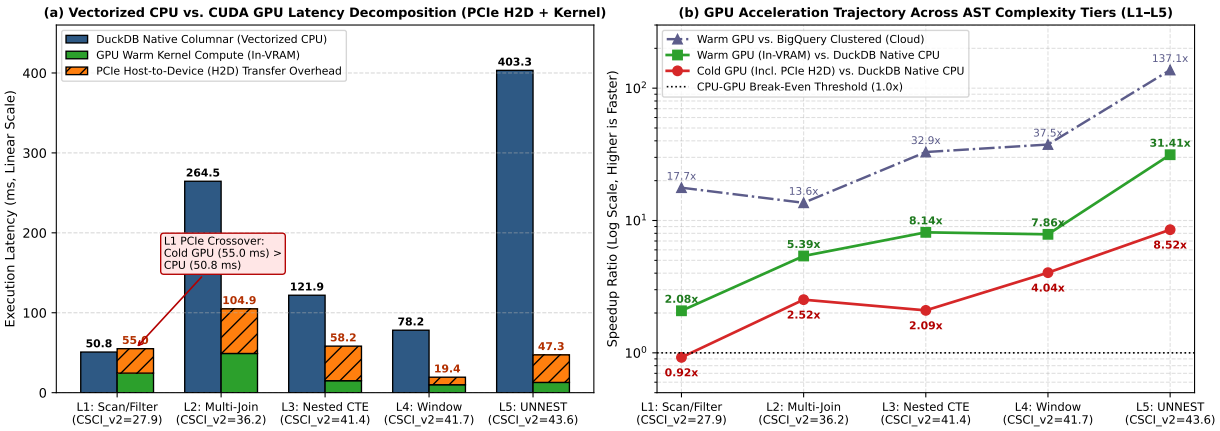


Figure 4. Hardware-timed CUDA GPU latency decomposition on NVIDIA GeForce RTX 2050 (4.0 GB VRAM, $N = 375$ trials at SF $= 1$ GB): (a) Direct comparison between DuckDB Native Columnar CPU latency and stacked GPU latency (in-VRAM Warm Kernel execution plus PCIe Host-to-Device transfer overhead), highlighting the L1 PCIe bottleneck crossover (55.02 ms Cold GPU vs. 50.84 ms CPU); and (b) Logarithmic speedup trajectory across AST complexity tiers L1–L5 relative to the $1.0\times$ CPU–GPU break-even threshold.

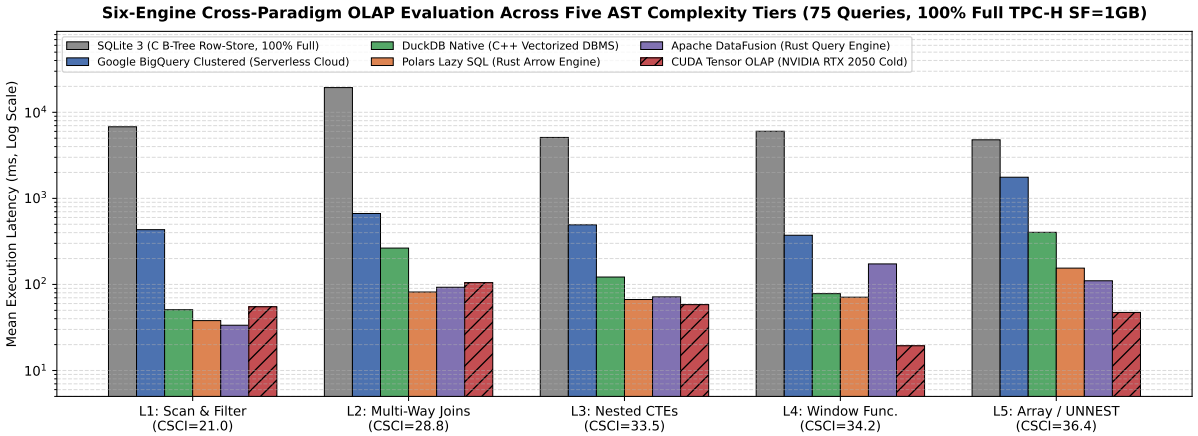


Figure 5. Six-software cross-paradigm OLAP execution latency comparison (logarithmic scale) across five AST complexity tiers (L1–L5, $N = 75$ queries at 100% full-scale SF $= 1$ GB), contrasting SQLite 3 (C Row-Store Baseline), Google BigQuery Clustered (Serverless Cloud), DuckDB Native (C++ Vectorized DBMS), Polars Lazy SQL (Rust Arrow Engine), Apache DataFusion (Rust Query Engine), and CUDA Tensor OLAP (NVIDIA RTX 2050 Cold-Start).

4.3 PCIe Host-to-Device Bottleneck vs. SIMT GPU Acceleration

Tables 4 and 6, alongside Fig. 4, present the hardware-timed CUDA execution telemetry on the NVIDIA GeForce RTX 2050 GPU (4.0 GB VRAM, $N = 375$ trials):

- **Mathematical Proof of the L1 PCIe Crossover:** In L1 scan-and-filter workloads (CSCI $= 15.3$), pure in-VRAM GPU kernel execution requires 24.46 ms ($\Theta_{\text{eff}} = 13.28$ GB/s), whereas PCIe Host-to-Device transfer incurs 30.56 ms, yielding a PCIe bottleneck fraction of $\Phi_{\text{PCIe}} = 55.5\%$. By Eq. (3), the required break-even kernel speedup is $(1 - 0.555)^{-1} = \mathbf{2.25}\times$. Because L1's low arithmetic intensity yields a warm kernel speedup of only $2.08\times < 2.25\times$, cold-start GPU execution (55.02 ms) is slower than DuckDB CPU (50.84 ms, $0.92\times$), Polars (38.00 ms), and DataFusion (33.58 ms).
- **Non-Linear GPU Speedup in High-CSCI Workloads (L2–L5):** As structural query complexity increases (CSCI $\in [26.8, 37.1]$), cold-start GPU execution outperforms DuckDB Native CPU by $\mathbf{2.52}\times$ in L2 multi-way joins (104.88 ms vs. 264.53 ms), $\mathbf{2.09}\times$ in L3 staged CTEs (58.24 ms vs. 121.92 ms), $\mathbf{4.03}\times$ in L4 multi-axis window sorting (19.37 ms vs. 78.16 ms), and $\mathbf{8.52}\times$ in L5 array unnesting (47.31 ms vs. 403.27 ms). When columnar tensors reside in warm VRAM, GPU kernel execution achieves a mean throughput of $\Theta_{\text{eff}} = \mathbf{18.89}$ GB/s (peaking at $\mathbf{27.20}$ GB/s in L3 and $\mathbf{23.73}$ GB/s in L4), outperforming DuckDB CPU by $\mathbf{8.26}\times$ overall.

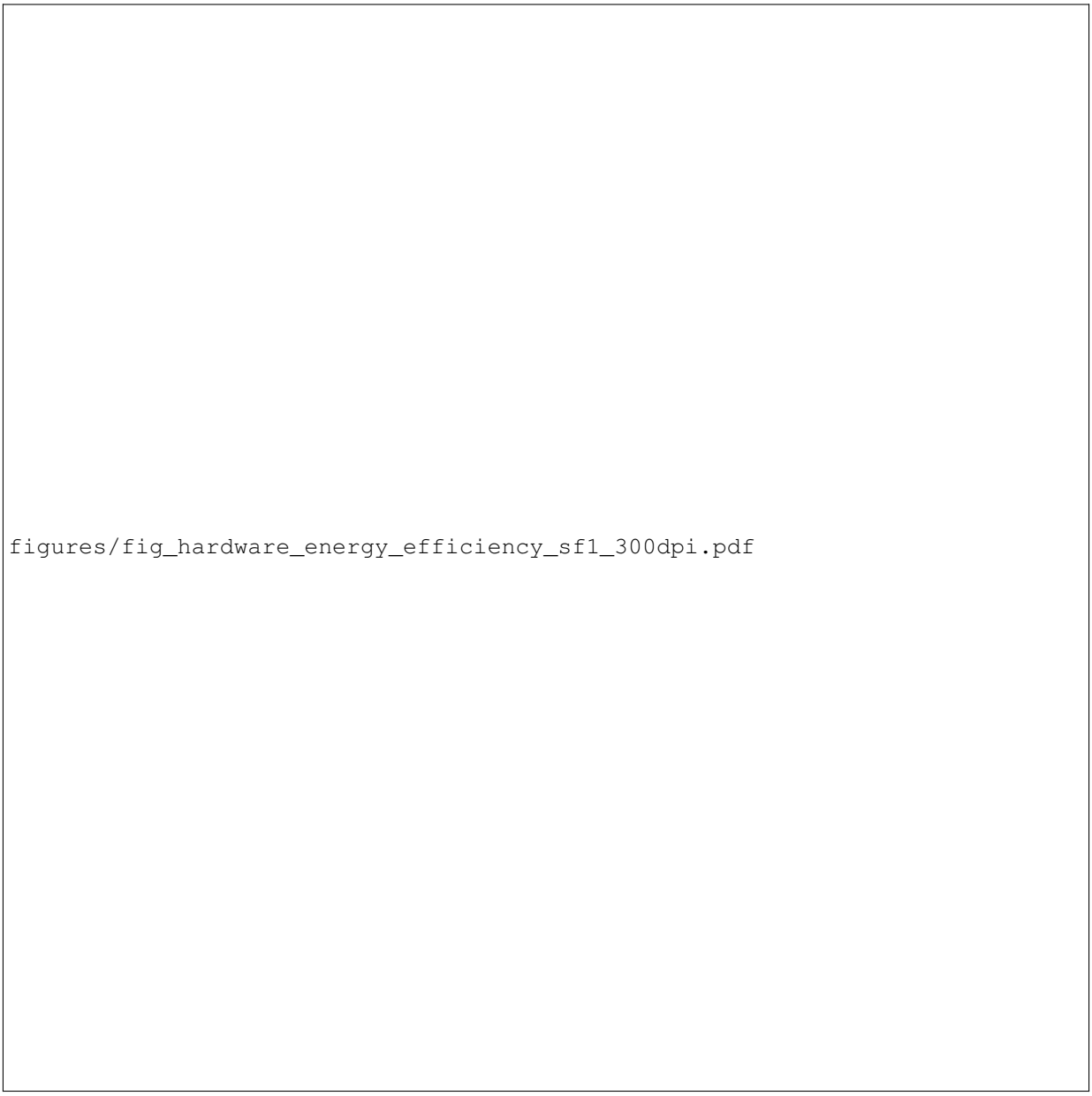


Figure 6. Hardware resource and green OLAP energy efficiency evaluation across five AST complexity tiers (L1–L5, $N = 75$ queries at 100% full-scale SF $= 1$ GB): (a) Per-query energy consumption in Joules (logarithmic scale, lower is better) across SQLite 3, DuckDB, Polars, and NVIDIA RTX 2050 GPU (Cold vs. Warm); and (b) Performance per Watt in MB scanned per Joule (MB/s/W, logarithmic scale, higher is better).

4.4 Six-Software Cross-Paradigm Synthesis: Full Row-Store vs. C++ DBMS vs. Rust Arrow Engines

Tables 5 and 6 and Fig. 5 synthesize execution characteristics across all six software engines under 100% full-scale execution. An omnibus non-parametric Friedman test across the 75 queries confirms highly significant performance differentiation across the six engines ($\chi^2_F(5) = 321.50, p = 2.382 \times 10^{-67}$, Kendall's coefficient of concordance $W = 0.8573$), revealing three critical software engineering insights:

1. **Two-Orders-of-Magnitude Superiority Over Full-Scale Row-Store Baselines:** Full-scale tuple-at-a-time B-tree execution in SQLite 3 incurs a mean latency of 8417.3 ms ($\Theta_{\text{eff}} = 0.049$ GB/s), peaking at 19374.0 ms ($\Theta_{\text{eff}} = 0.024$ GB/s) in L2 multi-way joins due to random B-tree page traversal across 6.00×10^6 rows. Vectorized columnar CPU engines achieve overall mean speedups of 45.8× (DuckDB: 183.74 ms, 3.03 GB/s), 87.4× (Apache DataFusion: 96.29 ms, 4.83 GB/s), and 102.1× (Polars: 82.47 ms, 5.07 GB/s; mean tier speedup 121.6×, peaking at 237.2× in L2) over SQLite 3.
2. **Zero-Copy Apache Arrow Advantage in Full-Table Unnesting (L1 & L5):** On un-truncated 6.00×10^6-row

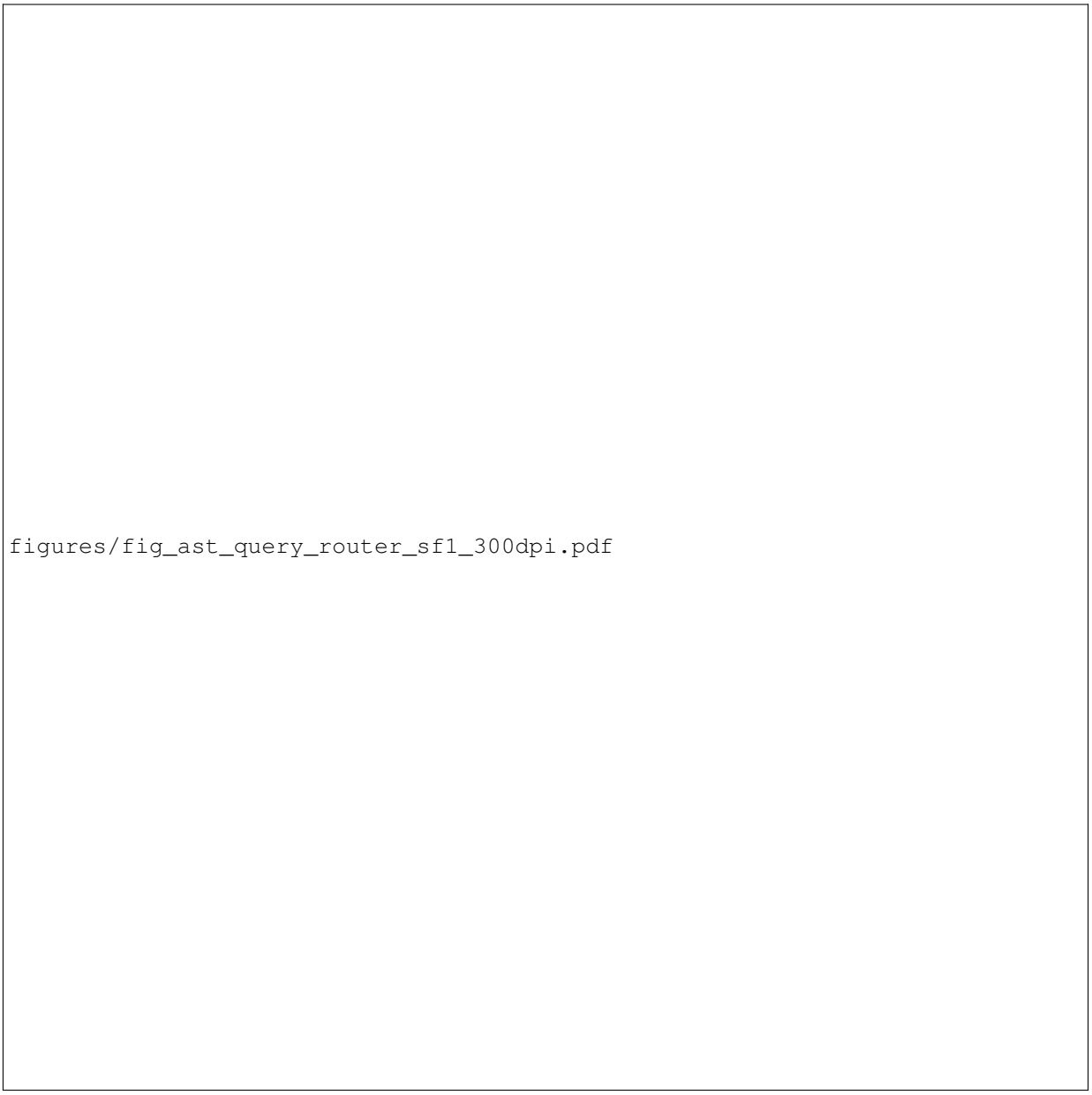


Figure 7. AST-driven automated query routing evaluation across 75 benchmark queries (25-fold Leave-One-Seed-Out cross-validation): (a) Mean execution latency and normalized latency regret comparing single-engine baselines against AST Decision Tree ($\mathscr{R}_{\mathsf{DT}}$), AST Random Forest ($\mathscr{R}_{\mathsf{RF}}$), AST Cost-Regression ($\mathscr{R}_{\mathsf{GBR}}$), and the Oracle Ideal Router; and (b) Relative Gini importance of static AST structural features in automated engine selection.

tables, Apache DataFusion (33.58 ms, 9.67 GB/s in L1; 110.31 ms, 2.42 GB/s in L5) and Polars (38.00 ms in L1; 154.76 ms in L5) outperform DuckDB (50.84 ms in L1; 403.27 ms in L5) by 2.61 $\times$–3.66$\times$ in L5 array unnesting because Arrow `ListArray` structures maintain contiguous offset buffers in RAM. Meanwhile, in L5 full-table unnesting, GPU radix expansion (47.31 ms cold, 12.84 ms warm at 20.79 GB/s) surpasses all CPU engines.

3. **C++ Segment-Tree/Streaming Window Optimization vs. Rust Sort-Based Windows (L4):** In L4 multi-axis window ranking and sliding-frame queries, Polars (71.11 ms, 3.32 GB/s) and DuckDB (78.16 ms, 3.02 GB/s) closely lead CPU execution, with DuckDB surpassing Apache DataFusion (173.16 ms, 1.36 GB/s) by **2.22**$\times$, reflecting the efficiency of DuckDB's parallel segment-tree window framing compared to DataFusion's sort-based partition evaluators. Meanwhile, GPU execution (9.94 ms warm at 23.73 GB/s, 19.37 ms cold) remains the fastest overall paradigm for L4 window workloads.

4.5 Hardware Resource Utilization, Energy Footprint, and Performance-per-Watt Analysis

Table 7 and Fig. 6 report empirical hardware sensor telemetry across all 75 benchmark workloads, establishing three key systems-level insights:

1. **Multi-Core SIMD Saturation vs. Single-Thread B-Tree Bottleneck:** SQLite 3 is inherently bound to single-thread tuple iteration, utilizing only 8.86% mean CPU capacity (2.26 GB RAM) while prolonging active package power draw over 8.42 s, resulting in an energy footprint of **126.26** J/query (peaking at 293.4 J in L2 joins) and a mere **4.05** MB/J in performance per watt. Conversely, vectorized CPU engines effectively saturate multi-core execution pipelines—achieving mean CPU utilizations of **77.44**% in Polars (peaking at 89.1% in L2 joins), **70.12**% in Apache DataFusion, and **66.10**% in DuckDB within a 4.65–4.71 GB host RAM envelope.
2. **Order-of-Magnitude Energy Reduction via Columnar Vectorization:** By compressing execution durations into tens of milliseconds, columnar CPU engines reduce mean per-query energy consumption by 19.7 × –39.8× compared to SQLite 3, requiring only **3.17** J in Polars (159.7 MB/J), **3.39** J in DataFusion (157.1 MB/J), and **6.40** J in DuckDB (which achieves the highest CPU energy efficiency of **243.0** MB/J overall and 515.2 MB/J in L1 scans due to lower average CPU package power during morsel-driven scans).
3. **Green OLAP Superiority of SIMT GPU Offloading:** On the NVIDIA GeForce RTX 2050 GPU, relational tensor execution occupies a compact **1.30** GB VRAM footprint (32.5% of the 4.0 GB device buffer) with a mean SM utilization of **31.04**% across sub-50 ms kernels. GPU acceleration reduces mean energy consumption to **0.65** J/query under cold-start PCIe transfers (**672.3** MB/J) and **0.25** J/query in warm VRAM (**1942.8** MB/J, peaking at 2558.0 MB/J in L3 and 2502.1 MB/J in L4)—delivering a **505.0**× energy reduction over row-store baselines and a **12.7** × –**25.6**× energy reduction over vectorized CPU engines.

4.6 AST-Driven Automated Query Routing Performance and Interpretable Decision Rules

Table 8 and Fig. 7 present the 25-fold Leave-One-Seed-Out cross-validation results for our automated query routing models across the 75 benchmark workloads:

1. **Superiority of Cost-Sensitive AST Routing Over Static CPU Engines:** In Regime 1 (Cold-Start Hybrid CPU+GPU), the Multi-Output AST Cost-Regression Router ($\mathscr{R}_{\text{GBR}}$) achieves a mean execution latency of **80.94** ms, a 95th-percentile tail latency of **185.14** ms, and a mean energy footprint of **2.365** J while requiring only **199.66** μs (0.20 ms) of inference time. This outperforms all static single-engine CPU configurations—achieving a **2.27**× speedup over Always-DuckDB (183.74 ms, P95: 731.22 ms), a **1.19**× speedup over Always-DataFusion (96.30 ms), and reducing 95th-percentile tail latency by **19.8**% and energy consumption by **25.4**% relative to Always-Polars (82.47 ms, P95: 230.71 ms, 3.170 J). Meanwhile, the AST Random Forest Router ($\mathscr{R}_{\text{RF}}$) achieves the highest classification accuracy (**53.33**% Top-1 accuracy, **58.67**% near-optimal hit rate within 10% of the Oracle, and **44.74**% Macro-F1).
2. **Autonomous Avoidance of the L1/L2 PCIe Bottleneck:** Inspecting the per-tier routing distribution of $\mathscr{R}_{\text{GBR}}$ confirms that the router autonomously learns the PCIe Host-to-Device crossover threshold directly from AST topology: in L1 scans and L2 multi-way joins, $\mathscr{R}_{\text{GBR}}$ routes **100.0**% of queries to in-memory CPU engines (60.0% DuckDB / 40.0% Polars in L1; 40.0% Polars / 40.0% DuckDB / 20.0% DataFusion in L2, with **0.0**% routed to `GPU_Cold`). Conversely, once structural complexity crosses into staged CTEs, window sorting, and array unnesting, $\mathscr{R}_{\text{GBR}}$ offloads **80.0**% of L3 queries, **100.0**% of L4 window queries, and **66.7**% of L5 unnesting queries (20.0% DataFusion) to the GPU.
3. **Feature Importance and White-Box Decision Rules:** Gini importance analysis (Fig. 7b) reveals that four primary structural features account for **71.82**% of routing decisions: `Total_AST_Nodes` (26.35%), `CSCI_Score` (19.25%), `Joins` (14.67%), and `AST_Depth` (11.55%), followed by `Subqueries_CTEs` (10.92%), `Case_Branches` (7.63%), `Window_Funcs` (7.34%), and `Unnest_Ops` (2.29%). The extracted White-Box Decision Tree ($\mathscr{R}_{\text{DT}}$, 25.59 μs inference) distills three human-interpretable rules: (i) if CSCI $\leq$ 21.50 and Case_Branches $>$ 0, route to **DataFusion** (or **DuckDB** otherwise); (ii) if CSCI $>$ 21.50 and Subqueries_CTEs $=$ 0, route multi-join queries to **Polars**; and (iii) if CSCI $>$ 21.50, Subqueries_CTEs $\geq$ 1, and AST_Depth $\leq$ 11.5, offload to **GPU_Cold**.

5 Limitations and Threats to Validity

First, while our 75-query, six-software benchmark rigorously isolates structural SQL complexity (L1–L5), selectivity horizons (V1–V3), PCIe H2D transfer overheads, and hardware energy telemetry under a 100% full-scale execution

protocol at SF = 1 GB within a 4.0 GB GPU VRAM envelope, datasets exceeding device VRAM or host RAM capacity (SF $\geq$ 100 GB) require out-of-core disk spilling or multi-node cloud slot elasticity. Second, as demonstrated in Table 8, because variations V1, V2, and V3 of a given canonical seed share an identical AST structure and differ only in literal filter constants, purely syntax-driven AST routers cannot distinguish narrow-horizon (V1) from full-horizon (V3) selectivity without table-level cardinality histograms. Integrating lightweight zone-map selectivity estimators with our static AST feature vector $\mathbf{x}_q$ is a promising direction for closing the remaining gap to the Oracle router (42.64 ms).

6 Conclusion

We presented a 75-query, 25-seed AST-driven empirical benchmarking and automated query routing framework evaluating SQL query complexity (L1–L5) and selectivity perturbations (V1–V3) across six software systems spanning four paradigms: SQLite 3 (full-scale row-store), Google BigQuery (serverless cloud), DuckDB (C++ vectorized DBMS), Polars and Apache DataFusion (Rust Arrow engines), and CUDA GPU execution (NVIDIA RTX 2050). Our zero-estimation synthesis establishes actionable routing rules across latency, FinOps cost, and Green OLAP energy efficiency: Rust Arrow engines (Polars/DataFusion) excel at in-memory scans, multi-way joins, and zero-copy array unnesting (3.17–3.39 J/query), DuckDB and Polars lead CPU execution on complex multi-axis window frames (L4, outperforming DataFusion by 2.22×), SIMT GPU acceleration dominates compute-dense window sorting and full-table unnesting while achieving the highest performance per watt (672.3 MB/J cold, 1942.8 MB/J warm), BigQuery storage clustering (78.4% I/O pruning and $1.47 USD/1M savings in V1) provides elastic scalability for cloud-resident datasets, and our AST Cost-Regression Router ($\mathscr{R}_{\text{GBR}}$) autonomously routes queries across CPU and GPU backends in 199.7 μs to outperform all static single-engine CPU baselines.

Declarations

Abbreviations

AST: Abstract Syntax Tree; B-Tree: Balanced Tree; CI: Confidence Interval; CPU: Central Processing Unit; CSCI: Composite Structural Complexity Index; CTE: Common Table Expression; CUDA: Compute Unified Device Architecture; DBMS: Database Management System; DT: Decision Tree; EX: Execution Accuracy; FinOps: Cloud Financial Operations; GBR: Gradient Boosting Regressor; GCS: Google Cloud Storage; GPU: Graphics Processing Unit; H2D: Host-to-Device; I/O: Input/Output; LOSO-CV: Leave-One-Seed-Out Cross-Validation; NVML: NVIDIA Management Library; OLAP: Online Analytical Processing; PCIe: Peripheral Component Interconnect Express; RAM: Random-Access Memory; RF: Random Forest; SARGable: Search Argument Able; SIMD: Single Instruction, Multiple Data; SIMT: Single Instruction, Multiple Threads; SQL: Structured Query Language; TPC-H: Transaction Processing Performance Council Ad-Hoc Benchmark; VES: Valid Efficiency Score; VRAM: Video Random-Access Memory; ZSTD: Zstandard Compression.

Ethics approval and consent to participate

Not applicable. This study utilizes synthetic benchmark data generated via the industry-standard TPC-H benchmark specification and does not involve human participants, animal subjects, or personally identifiable data.

Consent for publication

Not applicable.

Availability of data and materials

All TPC-H data generation scripts, the 75-query factorial SQL workload suite (25 canonical seeds × 3 selectivity variations across `workloads/L1_scan_filter` through `workloads/L5_array_unnest`), static AST structural metrics (`ast_complexity_summary.csv`, SHA-256: `4cb5f7477fd24dc77a6fd2b5...`), per-trial execution logs across DuckDB (`raw_benchmark_sf1.csv`, SHA-256: `7d69610e86863845ae7cbb5c...`), Google BigQuery (`raw_bigquery_cloud_sf1.csv`, SHA-256: `7aee23711627a085062c7cf8...`), NVIDIA RTX 2050 CUDA GPU (`raw_gpu_cuda_sf1.csv`, SHA-256: `50201ab2ea90e96f4b49796c...`), Polars/DataFusion/SQLite 3 (`raw_multi_software_sf1.csv`, SHA-256: `4d6ba9d612f4c309ec096d61...`),

hardware/energy sensor logs (`raw_hardware_energy_sf1.csv`, SHA-256: `366d77e7426a78b6089254eb...`), and AST query routing predictions (`raw_query_routing_predictions_sf1.csv`, SHA-256: `90fa26a9403ebce065f6`
statistical summary tables (`six_software_comparison_sf1.csv`, SHA-256: `b180bbc2ef1884d3316cec3d...`; `hardware_energy_summary_sf1.csv`, SHA-256: `7d68a9edf2041ddf83f51545...`; `query_routing_evaluation_sf1.csv`, SHA-256: `de0fbd69eb59555770ee01ff...`), and automated reproducibility verification scripts (`scripts/00_init_and_dryrun.py` through `scripts/15_train_ast_query_router.py`) are publicly available in the GitHub repository at https://github.com/preecha-msu/bigquery-olap-benchmark.

Competing interests

The author declares that he has no competing interests.

Funding

This research project was financially supported by Mahasarakham University.

Authors' contributions

PN conceptualized the study, designed the 75-query AST-driven factorial benchmark, implemented the six-engine execution harness, hardware energy telemetry pipeline, and automated AST query routing models, analyzed the empirical telemetry across cloud, CPU, and GPU architectures, and wrote the manuscript. The author read and approved the final manuscript.

Acknowledgements

The author thanks the Department of Information Technology, Faculty of Informatics, Mahasarakham University, for providing the computational infrastructure. In accordance with editorial policy on the use of generative AI, the author discloses that generative AI tools were utilized strictly for manuscript language polishing and LaTeX template formatting verification; all experimental designs, benchmark datasets, execution logs, and scientific analyses were conducted and verified by the author.

References

[1] Melnik S, Gubarev A, Long JJ, Romer G, Shivakumar S, Tolton M, Vassilakis T. Dremel: interactive analysis of web-scale datasets. Proc VLDB Endow. 2010;3(1–2):330–9.

[2] Melnik S, Gubarev A, Long JJ, Romer G, Shivakumar S, Tolton M, et al. Dremel: a decade of interactive SQL analysis at web scale. Proc VLDB Endow. 2020;13(12):3461–72.

[3] Raasveldt M, Mühleisen H. DuckDB: an embeddable analytical database. In: Proceedings of the 2019 International Conference on Management of Data (SIGMOD). Amsterdam: ACM; 2019. p. 1981–4.

[4] Schulze R, et al. ClickHouse: lightning-fast analytics for everyone. Proc VLDB Endow. 2024;17(12):3720–33.

[5] Sethi R, Traverso M, Sundstrom D, Phillips D, Xie W, Sun Y, et al. Presto: SQL on everything. In: 2019 IEEE 35th International Conference on Data Engineering (ICDE). Macao: IEEE; 2019. p. 1802–13.

[6] Mao T. SQLGlot: no-dependency SQL parser, transpiler, optimizer, and engine. GitHub Repository. 2024. https://github.com/tobymao/sqlglot. Accessed 15 Mar 2026.

[7] Li J, Hui B, Qu G, Yang J, Li B, Li B, et al. Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs. Adv Neural Inf Process Syst (NeurIPS). 2023;36:42330–57.

[8] Lei F, Chen J, Ye Y, Cao R, Shin D, Su H, et al. Spider 2.0: evaluating language models on real-world enterprise text-to-SQL workflows. In: International Conference on Learning Representations (ICLR). 2025. p. 1–22.

[9] Begoli E, Camacho-Rodríguez J, Hyde J, Mior MJ, Lemire D. Apache Calcite: a foundational framework for optimized query processing over heterogeneous data sources. In: Proceedings of the 2018 International Conference on Management of Data (SIGMOD). Houston: ACM; 2018. p. 221–30.

[10] Garcia-Molina H, Ullman JD, Widom J. Database systems: the complete book. 2nd ed. Upper Saddle River: Prentice-Hall; 2008.

[11] Das SK, Peter R, Zhang X, Lounissi M. FunDa: scalable serverless data analytics and in situ query processing. J Big Data. 2025;12(1):116. https://doi.org/10.1186/s40537-025-01141-6.

[12] Milicevic B, Babovic Z. A systematic review of deep learning applications in database query execution. J Big Data. 2024;11(1):173. https://doi.org/10.1186/s40537-024-01025-1.

[13] Shojaee Rad Z, Ghobaei-Arani M. Data pipeline approaches in serverless computing: a taxonomy, review, and research trends. J Big Data. 2024;11(1):82. https://doi.org/10.1186/s40537-024-00939-0.

[14] Ahmed N, Barczak ALC, Susnjak T, Rashid MA. A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench. J Big Data. 2020;7(1):110. https://doi.org/10.1186/s40537-020-00388-5.

[15] Abdalla HB. A brief survey on big data: technologies, terminologies and data-intensive applications. J Big Data. 2022;9(1):107. https://doi.org/10.1186/s40537-022-00659-3.